Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul Calculatoare și Tehnologia Informației

Computer Architecture
Laborator 2: Verilog Combinațional

Ștefan-Dan Ciocîrlan · 2026-2027

Cuprins

1. Descriere comportamentală 1
1.1. Assign 1
1.2. Constante 2
1.3. Operatori 3
2. Parametrizarea modulelor 4
2.1. Parameter 4
2.2. Construirea și instanțierea modulelor parametrizabile 4
2.3. Macro: `define 5
3. Testare 5
3.1. Blocul initial 6
3.2. Sincronizarea prin întârziere 6
3.3. Afișare 6

1. Descriere comportamentală

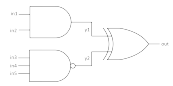
Laboratorul curent va prezenta elementele Verilog folosite pentru descrierea comportamentală. Aceasta poate descrie **ce face** circuitul și nu **cum** va fi acesta implementat. Mai mult, vom completa un modul funcțional cu un modul de testare, astfel încât să avem posibilitatea de a verifica implementarea pe care o concepem.

1.1. Assign

Deși partiționarea circuitelor în cadrul unei arhitecturii duce la simplificarea implementării unui modul, implementarea acestuia la nivel de porți logice este rareori folosită, întrucât aceasta devine complicată și dificil de înțeles.

Primul pas este reprezentat de ușurarea modalității de scriere a unei funcții logice. Pentru aceasta, Verilog oferă o instrucțiune numită **atribuire continuă**. Aceasta folosește cuvântul cheie ’’assign’’ și “atribuie” unei variabile de tip ’’wire’’, valoarea expresiei aflată în partea dreaptă a semnului egal. Atribuirea are loc la fiecare moment de timp, deci orice schimbare a valorii expresiei din partea dreaptă se va propaga imediat. În partea stângă a unei atribuiri continue se poate afla orice variabilă declarată de tip wire sau orice ieșire a modulului care nu are altă declarație (ex. reg). Expresiile din partea dreaptă pot fi formate din orice variabile sau porturi de intrare și de ieșire și orice operatori suportați de Verilog.

Bazându-ne pe circuitul descris în figura de mai jos, acesta se poate scrie sub formă de atribuiri continue în următoarea formă:

Simbol	Cod
	`wire y1, y2; xor(out, y1, y2); and(y1, in1, in2); nand(y2, in3, in4, in5);`

1.1.1. Exemplu atribuire continuă

```
module my_beautiful_module (
    output out,
    input  i1,i2,i3,i4,i5);

          assign y1 = i1 & i2;
          assign y2 = ~(i3 & i4 & i5);

          assign out = y1 ^ y2;

endmodule
```

sau, mai concis:

```
module my_beautiful_module (
    output out,
    input  i1,i2,i3,i4,i5);

          assign out = (i1 & i2) ^  (~(i3 & i4 & i5));

endmodule
```

Se poate observa că o atribuire continuă este mult mai ușor de scris, de înțeles și de modificat decât o descriere echivalentă bazată pe instanțierea de primitive. Circuitul descris de o atribuire continuă poate fi însă relativ ușor sintetizat ca o serie de porți logice care implementează expresia dorită, unii operatori având o corespondență directă cu o poartă logică.

Este o eroare să folosiți aceeași variabilă destinație pentru mai multe atribuiri continue. Ele vor încerca simultan să modifice variabila, lucru ce nu este posibil în hardware.

1.2. Constante

Pentru specificarea valorilor întregi este folosită următoarea sintaxă:

''[size]'[radix] constant_value''

- numerele conțin doar caracterele bazei lor și caracterul '_'
- pentru a ușura citirea, se poate folosi caracterul '_' ca delimitator
- caracterul '?' specifică impedanță mare (z)
- caracterul 'x' specifică valoare necunoscută
- se poate specifica dimensiunea numărului în biți dar și baza acestuia (b,B,d,D,h,H,o,O - binar, zecimal, hexa, octal)

```
8'b1;         //binar, pe 8 biti, echivalent cu 1 sau 8'b00000001
8'b1010_0111; //binar, echivalent cu 167 sau 8'b10100111
4'b10;        //binar, pe 4 biti, echivalent cu 2 sau 4'b0010 etc.
126;          //scriere in decimal
16'habcd;     //scriere in hexazecimal
```

1.3. Operatori

Descrierea comportamentală la nivelul fluxului de date, descrisă anterior, presupune în continuare cunoaşterea schemei hardware la nivelul porţilor logice sau, măcar, expresia logică. Deşi reprezintă o variantă mai simplă decât utilizarea primitivelor, nu este cea mai facilă.

Pentru a uşura implementarea, Verilog pune la dispoziţie mai multe tipuri de operatori. Unii dintre aceştia sunt cunoscuţi din limbajele de programare precum C, C++, Java, şi au aceeaşi funcţionalitate. Alţii sunt specifici limbajului Verilog şi sunt folosiţi în special pentru a descrie uşor circuite logice. Cu ajutorul acestora putem simplifica implementarea, apelând la construcţii folosind limbajul de nivel înalt.

Tabelul de mai jos conţine operatorii suportaţi de Verilog, împreună cu nivelul lor de precedenţă.

Simbol	Funcţie	Precedenţă
`! ~ + -` (unari)	Complement, Semn	1
`**`	Ridicare la putere	2
`* / %`	Înmulţire, Împărţire, Modulo	3
`+ -` (binari)	Adunare, Scădere	4
`<< >> <<< >>>`	Shiftare	5
`< <= > >= == !=`	Relaţionali	6
`& ~& ^ ~^ ^~ | |`	Reducere	7
`&&`	Logici	8.1
`||`	Logici	8.2
`?:`	Condiţional	9
`{,}`	Concatenare	

În continuare sunt prezentaţi operatorii mai neobişnuiţi suportaţi de Verilog:

- Operatorii de shiftare aritmetică; realizează shiftarea cu păstrarea bitului de semn, pentru variabilele declarate ca fiind cu semn.

```
wire signed[7:0] a, x, y;
assign x = a >>> 1;  // dacă bitul de semn al lui a este 0 bitul nou
                     //introdus este 0
                     // dacă bitul de semn al lui a este 1 bitul nou
                     // introdus este 1
assign y = a <<< 1;  // bitul nou introdus este tot timpul 0,
                     //asemănător cu operatorul <<
```

- Operatorii de reducere; se aplică pe un semnal de mai mulţi biţi şi realizează operaţia logică între toţi biţii semnalului

```
wire[7:0] a;
wire x, y, z;
assign x = &a;  // realizeaza AND între toţi biţii lui a
assign y = ~&a; // realizează NAND între toţi biţii lui a
assign z = ~^a; // realizeaza XNOR între toţi biţii lui a,
                // echivalent cu ^~
```

- Operatorul de concatenare; realizează concatenarea a două sau mai multe semnale, într-un semnal de lăţime mai mare.

```
wire[3:0] a, b;
wire[9:0] x;

// biţii 9:6 din x vor fi egali cu biţii 3:0 ai lui b
```

```
    // biții 5:4 din x vor fi egali cu 01
    // biții 3:2 din x vor fi egali cu biții 2:1 ai lui a
    // biții 1:0 din x vor fi egali cu 00
    assign x = {b, 2'b01, a[2:1], 2'b00};
```

2. Parametrizarea modulelor

2.1. Parameter

Cuvântul rezervat ’’parameter’’ este o construcție de limbaj în Verilog care permite unui modul să fie reutilizat cu specificații diferite. Spre exemplu, un sumator poate fi parametrizat să accepte o valoare pentru numărul de biți care poate să fie configurată diferit de la o simulare la alta. Comportamentul lor este similar cu cel al argumentelor unor funcții în alte limbaje de programare cunoscute. Folosind ’’parameter’’, este declarată o valoare constantă, prin urmare, este ilegală modificarea valorii acesteia în timpul simulării. De asemenea, este ilegal ca un alt tip de dată să aibă același nume ca unul dintre parametri.

```
parameter MSB = 7;       // MSB este un parametru cu valoarea constantă 7
parameter [7:0] number = 2’b11;    // o valoare de 2 biți este convertită
                                   // într-o valoare de 8 biți
```

O variabilă de tip parametru este vizibilă local, în modulul ce a fost declarată.

2.2. Construirea şi instanţierea modulelor parametrizabile

Instanțierea modulelor a fost folosită și în laboratorul anterior pentru a invoca logica implementată într-un alt modul. În acel context, era necesar să cunoaștem dimensiunea semnalelor din interfață pentru a le potrivi cu variabilele conectate la instanță. În cazul în care un modul are dimensiunile porturilor parametrizate, acesta poate fi instanțiat cu valori particulare ale parametrilor (diferite de cele predefinite). Să considerăm drept exemplu modulul de mai jos:

```
module my_beautiful_module (out, a, b);
    output [7:0] out;
    input [3:0] a;
    input [4:0] b;

    …// some logic
endmodule
```

Pentru a instanția acest modul, vom avea nevoie de 3 variabile de 8, 4, respectiv 5 fire pe care le vom conecta astfel:

```
My_beautiful_module inst1(out, a, b);
```

Pe de altă parte, având modulul:

```
module my_beautiful_parameterized_module(out, a, b);
    parameter a_width = 4;
    parameter b_width = 5;
    parameter out_width = 8;

    output [out_width-1:0] out;
    input [a_width-1:0] a;
    input [b_width-1:0] b;

    …// some logic
endmodule
```

Îi putem utiliza logica fără a depinde de o dimensiune predefinită a semnalelor din interfață

```
wire [4:0] out1;
wire [4:0] out2;
wire [2:0] a;
wire [1:0] b;

my_beautiful_parameterized_module #(.a_width(3),
                                    .b_width(2),
                                    .out_width(5)) inst2(out, a, b);

// Sau, menținându-se ordinea parametrilor, doar prin specificarea noilor dimensiuni:

my_beautiful_parameterized_module #(3, 2, 5) inst3(out, a, b);
```

2.3. Macro: `define

Un macro este un nume căruia i se poate asocia o valoare înainte de compilarea codului. Macro-urile sunt utile pe post de aliasuri, fără a utiliza resursele compilatorului. Acestea nu sunt variabile, prin urmare nu pot fi atribuite valori unui macro în timpul simulării. Majoritatea limbajelor de programare, inclusiv Verilog suportă definirea de macrouri.

În Verilog, un macro este specificat cu ajutorul directivei de compilator **`define**. Aceasta înlocuiește textul definit cu o valoare specifică. Un nume de tip `define este un macro global, însemnând că dacă este declarat într-un modul, va rămâne declarat și la ieșirea din modul. După ce macroul este declarat, poate fi apelat în cod cu ajutorul caracterului` (back-tic). Indiferent dacă sunt declarate în interiorul sau în afara unui modul, compilatorul le tratează la fel.

```
`define MY_NUMBER 5
`define MY_STRING “Hello world!”
`define ADD2PLUS2 2 + 2
```

3. Testare

Pentru testarea unui modul folosind simulatorul se creează module speciale de test, în care, printre altele, se vor atribui valori intrărilor. Simularea permite detecția rapidă a erorilor de implementare și corectarea acestora.

Pentru a crea un modul de test și a-l simula puteți urma tutorialul de simulare aici, iar această secțiune va prezenta câteva din construcțiile de limbaj pe care le puteți folosi într-un astfel de modul.

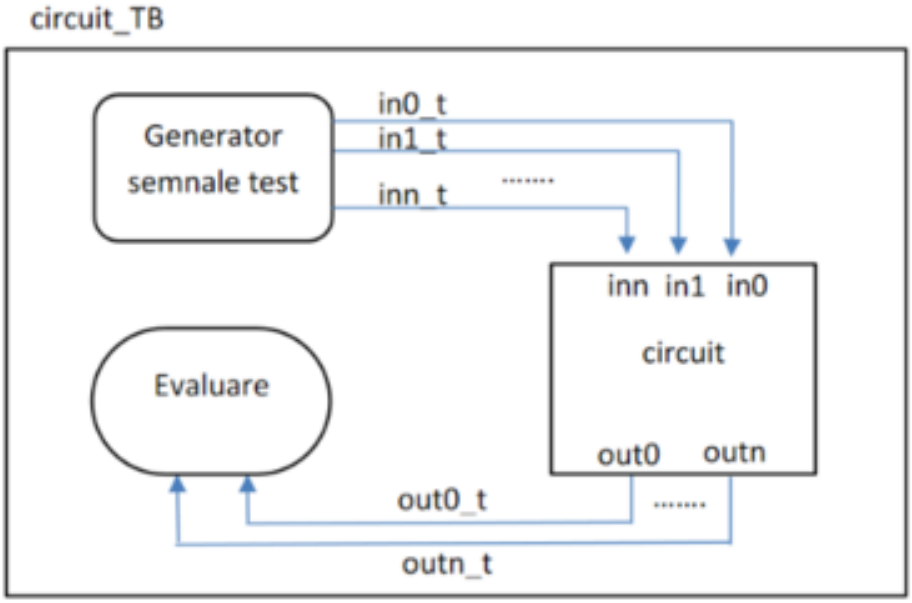


Figura 1: Diagrama testare circuit

3.1. Blocul initial

Blocurile *initial* descriu un comportament executat o singură dată la începerea/activarea simulării și sunt folosite pentru inițializări și în module de test. Instrucțiunile sale trebuie încadrate între cuvintele cheie *begin* și *end* și sunt executate secvențial.

```
initial begin
    a = 0;
    b = 1;
    #10; _ delay 10 unități de timp de simulare
    a = 1;
    b = 0;
end
```

Blocurile *initial* nu sunt sintetizabile, fiind folosite doar în simulări.

3.2. Sincronizarea prin întârziere

Folosind operatorul # se poate specifica o durată de timp între apariția instrucțiunii și momentul executării acesteia. Aceasta este utilă pentru a separa temporal diversele atribuiri ale intrărilor. Durata de timp este reprezentată prin unități de timp de simulare. De exemplu, dacă simularea folosește un *timescale* în nanosecunde, *#n* va reprezenta n nanosecunde.

3.3. Afișare

Atât în modulele de test cât și în modulele testate se pot folosi construcții pentru afișare în interiorul blocurilor *initial* și *always*. Una dintre aceste instrucțiuni este *display*:

```
$display(arguments);
```

Argumentele acestei comenzi sunt similare cu cele ale funcției *printf* din C, ca în exemplul de mai jos, iar specificația completă o puteți găsi aici. *$display* adaugă o linie nouă, iar dacă nu se dorește acest lucru se poate folosi comanda *$write*.

```
a = 1; b = 4;

$display("suma=%d", a+b);
```